GPU-Accelerated DMRG on AMD MI300X: Systematic Benchmarking and Lessons from Failed Optimizations

Yaakoub El Khamra[a,*]

[a], ,

Abstract

We present a systematic performance study of eleven density matrix renormalization group (DMRG) implementations, ranging from Python/CPU to hand-tuned GPU kernels in HIP/C++ targeting the AMD Instinct MI300X. The implementations span single-site and two-site DMRG, real-space parallel DMRG with domain decomposition, and an additive two-level parallel variant (A2DMRG), tested across three physics models: the Heisenberg spin-$\frac{1}{2}$ chain ($d = 2$, real), a Josephson junction array ($d = 5$, complex), and the transverse-field Ising model at criticality ($d = 2$, real). Benchmarks cover system sizes $L = 8$–128 and bond dimensions $\chi = 20$–256 on a single MI300X GPU with 192 GB HBM3.

The GPU implementations achieve speedups over optimized CPU code (quimb with OpenBLAS) that depend on both bond dimension and system size: at $\chi = 128$ the GPU wins 100% of short-chain ($L \leq 20$) configurations but only 50% at $L \geq 100$, while at $\chi \leq 50$ the CPU wins all medium-to-large systems ($L \geq 32$). Two-site DMRG converges 3.3× faster than single-site at large system sizes ($L = 100$, $\chi = 128$: 53 s vs. 177 s).

We report extensive negative results on algorithmic optimizations designed to exploit the MI300X's BLAS-3 throughput. Newton–Schulz iterative polar decomposition, intended to replace SVD with GEMM-heavy iterations, achieves 0% win rate across 50 configurations and diverges numerically at $\chi \geq 128$. Chebyshev-filtered subspace iteration, designed for computing many eigenvalues in density functional theory, is 1.9–11× slower than Lanczos for DMRG's single ground state. Cross-segment batched GEMM dispatch is slower in 18 of 19 configurations tested. Profiling reveals the root cause: CPU LAPACK SVD consumes 97–98% of per-sweep wall time at $\chi = 256$, and it-

*Corresponding author.

erative GPU replacements cannot match LAPACK's optimized QR-iteration algorithm at these matrix sizes. All source code, benchmark scripts, and raw data are publicly available.

Keywords: DMRG, GPU computing, tensor networks, AMD MI300X, performance benchmarking, matrix product states

---

1. Introduction

The density matrix renormalization group (DMRG) [1, 2] is the method of choice for computing ground states of one-dimensional quantum many-body systems. In the modern tensor network formulation [3, 4], DMRG variationally optimizes a matrix product state (MPS) by sweeping through the chain and solving local eigenvalue problems at each bond. The computational cost is dominated by tensor contractions (matrix–matrix multiplications) and singular value decompositions (SVDs), both of which scale as $O(\chi^3 d^2)$ per bond, where χ is the bond dimension and d is the local Hilbert space dimension.

Mature CPU implementations exist in several software packages, including ITensor [5] and quimb [6], which leverage optimized BLAS libraries for the tensor contractions. Graphics processing units (GPUs) are natural candidates for further accelerating DMRG, since the tensor contractions map directly to general matrix–matrix multiplication (GEMM), a workload for which modern GPUs achieve near-peak throughput. Prior work has explored GPU acceleration of DMRG and related tensor network methods [7], typically focusing on NVIDIA hardware with CUDA. The AMD Instinct MI300X, with 304 compute units and 192 GB of HBM3 memory providing 5.3 TB/s bandwidth, represents a new class of GPU hardware for scientific computing. Its matrix fused multiply-add (MFMA) units deliver 81.7 TFLOPS of FP64 matrix throughput, making it particularly attractive for the GEMM-dominated tensor contractions in DMRG.

However, the practical performance of GPU DMRG depends on a complex interplay of factors: the bond dimension χ (which determines GEMM sizes), the local dimension d (which determines batch counts), kernel launch overhead, host–device data transfer, and the SVD bottleneck. It is not obvious a priori at what problem sizes GPU acceleration becomes worthwhile, or whether algorithmic modifications designed to increase GEMM utilization will improve performance.

In this work, we present a systematic benchmarking study that addresses

these questions directly. We implement eleven DMRG variants—five CPU-based (including an MPI-parallel additive two-level variant) and six GPU-based—and benchmark them across three physics models at system sizes $L = 8$–128 and bond dimensions $\chi = 20$–256. Beyond the baseline GPU implementations, we investigate several algorithmic modifications specifically designed to exploit the MI300X's BLAS-3 throughput:

- Newton–Schulz iterative polar decomposition to replace SVD with GEMM iterations,
- Block-Davidson eigensolver to replace Lanczos with BLAS-3 subspace projections,
- Chebyshev-filtered subspace iteration as an alternative eigensolver,
- cross-segment batched GEMM dispatch for parallel DMRG.

All of these optimizations failed to improve performance at the tested bond dimensions. We analyze each failure in detail, as these negative results are arguably more informative for the community than the positive results: they delineate the boundary between regimes where iterative GPU methods can and cannot compete with optimized CPU linear algebra.

The paper is organized as follows. Section 2 describes the DMRG algorithm and its GPU implementation. Section 3 details the algorithmic variants and their motivation. Section 4 specifies the hardware, software, and benchmark methodology. Section 5 presents the performance results. Section 6 analyzes the negative results and identifies the SVD bottleneck. Section 7 summarizes our findings.

2. Methods

2.1. Tensor trains and matrix product states

We consider a one-dimensional quantum lattice of L sites with open boundary conditions. The Hilbert space has dimension d^L, where d is the local basis size (e.g., $d = 2$ for spin-$\frac{1}{2}$), making exact diagonalization intractable beyond $L \approx 20$. The key insight enabling DMRG is the *area law* of entanglement [8]: ground states of gapped local Hamiltonians in one dimension have entanglement entropy bounded by a constant independent of system size, which means they can be efficiently represented by matrix product states (MPS) with moderate bond dimension χ.

An MPS (equivalently, a tensor train in the numerical mathematics literature [9]) parametrizes the d^L-dimensional state vector as a product of L three-index tensors:

$$|\psi\rangle = \sum_{\sigma_1,\ldots,\sigma_L} A_1^{\sigma_1} A_2^{\sigma_2} \cdots A_L^{\sigma_L} \, |\sigma_1 \cdots \sigma_L\rangle, \tag{1}$$

where each $A_i^{\sigma_i}$ is a $\chi_{i-1} \times \chi_i$ matrix (with $\chi_0 = \chi_L = 1$) and $\sigma_i = 1, \ldots, d$ labels the local basis states. The bond dimension $\chi = \max_i \chi_i$ controls the expressiveness of the ansatz: $\chi = 1$ yields a product state, while $\chi = d^{L/2}$ can represent any state exactly. For ground states of local Hamiltonians, $\chi = O(\mathrm{poly}(L))$ typically suffices for fixed accuracy, giving an exponential compression from d^L to $O(L\chi^2 d)$ parameters.

2.2. Canonical forms

A central concept in MPS algorithms is the *canonical form* [3]. An MPS is in *left-canonical form* up to site k if each tensor A_i ($i < k$) satisfies $\sum_\sigma A_i^{\sigma\dagger} A_i^\sigma = I$, i.e., each tensor is a (partial) isometry when viewed as a matrix from left to right. Similarly, *right-canonical form* from site k means $\sum_\sigma A_i^\sigma A_i^{\sigma\dagger} = I$ for $i > k$. An MPS in *mixed-canonical form* centered at site k is left-canonical for $i < k$ and right-canonical for $i > k$, so that the tensor at site k (the "active" tensor) contains all the variational freedom.

Canonical form is established and maintained by QR or SVD decompositions. To left-canonicalize site i, reshape A_i into a $(\chi_{i-1}d) \times \chi_i$ matrix and compute the thin QR factorization $A_i = QR$; replace $A_i \leftarrow Q$ and absorb R into A_{i+1}. To truncate the bond dimension (as needed after two-site optimization), SVD is used instead: $A_i = USV^\dagger$, truncate to the $\chi_{\max}$ largest singular values, and split the factors between sites i and $i+1$. The truncation error $\epsilon = \sqrt{\sum_{k>\chi_{\max}} s_k^2}$ is bounded by the discarded singular values, providing a controlled approximation.

2.3. Matrix product operators

The Hamiltonian H is represented as a matrix product operator (MPO) [3, 4]:

$$H = \sum_{\boldsymbol{\sigma},\boldsymbol{\sigma}'} W_1^{\sigma_1\sigma_1'} W_2^{\sigma_2\sigma_2'} \cdots W_L^{\sigma_L\sigma_L'} \, |\boldsymbol{\sigma}\rangle\langle\boldsymbol{\sigma}'|, \tag{2}$$

where each $W_i^{\sigma_i\sigma_i'}$ is a $D_{i-1} \times D_i$ matrix and $D = \max_i D_i$ is the MPO bond dimension. For nearest-neighbor Hamiltonians, D is a small constant:

$D = 5$ for the Heisenberg model, $D = 3$ for the transverse-field Ising model, and $D = 4$ for the Josephson junction array. The MPO provides an exact, compact representation of the Hamiltonian that enables efficient computation of expectation values and H_{eff} applications in $O(\chi^2 dD)$ operations per bond.

2.4. The DMRG sweep

The density matrix renormalization group (DMRG) [1, 2] variationally optimizes the MPS by sweeping through the chain. In the modern formulation [3], a left-to-right sweep followed by a right-to-left sweep constitutes one "sweep" of the algorithm. At each bond b, the algorithm:

1. Forms the *effective Hamiltonian* H_{eff} by contracting the left environment L_{env} (encoding all sites $< b$), the MPO tensor(s) W_b, and the right environment R_{env} (encoding all sites $> b$) into a linear operator acting on the active tensor(s) at bond b.
2. Solves the *local eigenvalue problem*

$$H_{\text{eff}}|\theta\rangle = E|\theta\rangle \tag{3}$$

   for the ground-state energy E and optimal local tensor θ, using a Krylov eigensolver (Lanczos).
3. *Restores canonical form*: decomposes the updated tensor via SVD or QR and shifts the canonical center to the next bond.
4. *Updates the environment*: incorporates the newly optimized tensor into L_{env} (left sweep) or R_{env} (right sweep) for the next bond.

The environment tensors L_{env} and R_{env} are rank-3 tensors of dimension $\chi \times D \times \chi$ that encode the contraction of all MPS and MPO tensors to the left (or right) of the active bond. They are updated incrementally as the sweep progresses, at a cost of $O(\chi^2 dD + \chi^2 D^2)$ per bond. The total cost per sweep is $O(L\chi^3 d^2 D)$ for the eigensolve (dominated by the H_{eff} matrix–vector product) plus $O(L\chi^3 d)$ for the SVD/QR step.

In *single-site* DMRG, one MPS tensor is optimized at a time, giving an eigensolve on a vector of dimension χd. The bond dimension is fixed throughout the sweep, which is efficient but can trap the optimization in local minima if the initial bond dimension profile is poor.

2.5. Two-site DMRG

In two-site DMRG [10], pairs of adjacent MPS tensors are optimized simultaneously. The two-site tensor θ has dimensions $\chi_L d \times d\chi_R$, and after optimization it is split by SVD:

$$\theta = USV^\dagger, \tag{4}$$

where S is truncated to the $\chi_{\max}$ largest singular values. This allows the bond dimension to *adapt* during the sweep, which is critical for convergence from a random initial state: new Schmidt coefficients can appear that were absent in the initial MPS. The cost per bond is $O(\chi^2 d^2 D)$ for the eigensolve and $O(\chi^3 d^2)$ for the SVD, both scaling with d^2 rather than d compared to single-site.

We precompute fused MPO tensors $\widetilde{W}[b] = W_L[b] \otimes W_R[b]$ for each bond b, so the two-site H_{eff} application uses the same three-step GEMM pattern as single-site but with $d \to d^2$.

2.6. GPU implementation

All GPU implementations target the AMD MI300X via HIP/C++ and use rocBLAS [11] for BLAS operations. The code is templated on the scalar type, supporting both `double` (real) and `hipDoubleComplex` (complex) through a `ScalarTraits` dispatch layer.

The application of H_{eff} to a vector, the innermost operation of the Lanczos eigensolver, is implemented as three batched GEMM steps:

1. **Step 1**: $D \times d^2$ batched GEMMs contracting L_{env} slices with θ slices,
2. **Step 2**: One dense GEMM contracting the intermediate result with the fused MPO $\widetilde{W}$,
3. **Step 3**: $d^2 \times D$ batched GEMMs accumulating the contraction with R_{env} slices into the output.

The pointer arrays for batched GEMMs are computed by lightweight GPU kernels (typically < 64 threads), avoiding host-to-device DMA that would otherwise introduce pipeline stalls [12].

The Lanczos eigensolver runs entirely on the GPU using device-pointer mode for all rocBLAS reductions (`dot`, `nrm2`), eliminating implicit stream synchronizations. Only the final tridiagonal eigensolve (a $k \times k$ problem with $k \approx 15$–50) is performed on the CPU via LAPACK `dstev`. SVD is performed on the CPU via LAPACK `dgesvd` (QR-iteration based; or `zgesvd` for complex), as profiling showed this to be faster than GPU rocsolver for $\chi \leq 256$ (see Section 6.2).

2.7. Parallel DMRG

Our parallel DMRG implementation follows the real-space decomposition of Stoudenmire and White [13]. The L-site chain is partitioned into P segments, each assigned to an independent HIP stream and rocBLAS handle. The algorithm proceeds in three phases:

1. **Warmup**: Full-chain *single-site* DMRG sweeps on a single stream to establish an initial converged MPS at fixed bond dimension χ. Single-site is used because the eigensolver problem is smaller (χd vs. χd^2 for two-site), making warmup cheaper.
2. **Outer loop**: Parallel two-site segment sweeps (one `std::thread` per segment, each on its own HIP stream) using staggered directions, followed by boundary merge-and-optimize coupling via the Stoudenmire $V = \Lambda^{-1}$ prescription.
3. **Polish**: Full-chain *single-site* sweeps until convergence. Single-site is again used for the polish phase, as the bond dimension is already established and single-site sweeps are cheaper per bond.

The coupling step is essential: segment sweeps disrupt the global entanglement structure of the MPS, and only boundary coupling or full-chain resweeping restores the correct ground state. Experiments with boundary-only coupling sweeps (optimizing only sites near segment boundaries) failed to converge (energy errors > 8), confirming that coupling at every boundary is necessary.

3. Algorithmic variants

We implemented several algorithmic modifications designed to increase the fraction of computation performed as BLAS-3 (GEMM) operations on the GPU. The motivation was that profiling of the baseline GPU DMRG revealed CPU SVD as the dominant bottleneck (Section 6.2), and replacing it with GEMM-heavy iterative methods should exploit the MI300X's matrix throughput.

3.1. Newton–Schulz polar decomposition

The Newton–Schulz iteration computes the unitary polar factor U of a matrix $A = UP$ (where P is positive semidefinite) via the recurrence [14, 15]

$$X_0 = A/\|A\|_F, \qquad X_{k+1} = \tfrac{1}{2} X_k (3I - X_k^\dagger X_k). \tag{5}$$

This converges cubically to the unitary polar factor, with a maximum of 30 iterations (each consisting of two GEMMs) and convergence criterion $\|X^\dagger X - I\|_F < 10^{-10}$. After convergence, the singular values are obtained from the eigendecomposition of $P = U^\dagger A$, performed on the CPU.

The intent was to replace the LAPACK SVD call with ∼10–20 GPU GEMMs, shifting the computation from a BLAS-2-dominated factorization to pure BLAS-3. For single-site DMRG moving left-to-right, the MPS tensor θ is tall ($\chi_L d \times \chi_R$) and Newton–Schulz applies directly. For the reverse direction, θ is wide and we fall back to LAPACK SVD.

3.2. Block-Davidson eigensolver

The Block-Davidson method [16, 17] replaces Lanczos for the local eigenvalue problem (3). It maintains a subspace of block size $b = 4$ trial vectors, projects H_{eff} into the subspace using GEMM, solves the small projected eigenproblem via LAPACK `dsyev`, and expands the subspace with preconditioned residual corrections. The maximum subspace dimension is 32, after which the method restarts from the best eigenvector.

The motivation was that Lanczos is BLAS-1/2 dominated (one matrix–vector product plus dot products and axpy operations per iteration), while Davidson's subspace projection is a BLAS-3 operation (GEMM of the subspace basis with the H_{eff} application results).

3.3. Chebyshev-filtered subspace iteration

Chebyshev-filtered subspace iteration (CheFSI) [18] uses a polynomial filter to suppress unwanted eigenvalue components. Given spectral bounds $[\lambda_{\min}, \lambda_{\max}]$ from a short Lanczos run (10 steps), a degree-p Chebyshev polynomial T_p is applied via the three-term recurrence

$$y_{k+1} = 2\varphi(H)y_k - y_{k-1}, \qquad \varphi(H) = \frac{2H - (\lambda_{\max} + \lambda_{\min})I}{\lambda_{\max} - \lambda_{\min}}, \tag{6}$$

where each application of $\varphi(H)$ requires one H_{eff} matrix–vector product. The energy is extracted via the Rayleigh quotient after n_{outer} filter applications.

CheFSI was designed for density functional theory (DFT) calculations where many eigenvalues are needed simultaneously [18]. We tested it for DMRG's single ground state search with polynomial degree $p = 15$ and $n_{\text{outer}} = 20$.

3.4. Cross-segment batched GEMM

For parallel DMRG, we implemented a lock-step sweep mode that batches GEMM calls across segments. Instead of each segment independently dispatching GEMMs on its own stream, segments with matching bond dimensions are grouped and their GEMMs are dispatched as a single `rocblas_gemm_batched` call. This was intended to reduce per-kernel launch overhead by combining P small GEMMs into one batched dispatch.

3.5. Additive two-level parallel DMRG (A2DMRG)

We also implemented the additive two-level parallel DMRG (A2DMRG) algorithm of Grigori and Hassan [19] in Python with MPI parallelization. Unlike the real-space decomposition of Section 2.7, which partitions the chain into segments, A2DMRG updates *all* sites independently and combines the results via a coarse-space correction.

The algorithm proceeds in five phases per sweep:

1. **Orthogonal decompositions**: Construct d i-orthogonal MPS forms, one centered at each site.
2. **Parallel local micro-steps**: Solve the local eigenvalue problem (3) independently at every site, producing L candidate tensors. These are embarrassingly parallel across P processors.
3. **Coarse-space correction**: Form the $(d+1)\times(d+1)$ projected Hamiltonian and overlap matrices H_{coarse}, S_{coarse} in the subspace spanned by the d candidates plus the current MPS, and solve the generalized eigenvalue problem to find the optimal linear combination.
4. **Linear combination**: Assemble the updated MPS as the coarse-space eigenvector applied to the candidate tensors. This produces a bond dimension of up to $(L-1)\times\chi$.
5. **Compression**: Project back to the target bond dimension χ via TT-SVD (sequential truncated SVD from left to right).

The algorithm is an additive Schwarz method with coarse-space correction, analogous to the relationship between Jacobi (additive) and Gauss–Seidel (multiplicative) iterations for linear systems. Standard DMRG's sequential sweep is the multiplicative variant and converges faster per sweep, but the additive structure allows all local problems to be solved in parallel.

Our implementation includes three optimizations not in the original paper: (i) incremental environment caching (reducing the per-sweep cost from

$O(L^2\chi^2 D)$ to $O(L\chi^2 D)$), (ii) a pure numpy hot path replacing quimb tensor objects in the sweep phases, and (iii) configurable warm-up sweeps (serial DMRG2 via quimb) before entering the parallel A2DMRG loop.

To our knowledge, this is the first independent implementation of A2DMRG. The original paper [19] tested only quantum chemistry Hamiltonians (hydrogen chains H_6–H_{12}, C_2, N_2) with system sizes $d \leq 24$ using an in-house Julia implementation that is not publicly available. Our implementation extends the algorithm to lattice spin models (Heisenberg, Josephson, TFIM) and tests larger system sizes (L up to 64).

4. Computational details

4.1. Hardware and software

All GPU benchmarks were run on a single AMD Instinct MI300X accelerator (gfx942 architecture) with 304 compute units and 192 GB HBM3 memory providing 5.3 TB/s bandwidth. The peak FP64 throughput is 81.7 TFLOPS for matrix operations (MFMA instructions) and 40.9 TFLOPS for vector operations. The host CPU is an AMD EPYC processor.

The software environment consists of ROCm 7.2, rocBLAS (for GEMM), rocsolver (for GPU SVD, used in comparisons), and LAPACK via OpenBLAS 0.3.28 compiled from source. The system-provided OpenBLAS 0.3.20 was found to contain a bug in `dgesvd` that produced incorrect singular vectors; we compiled 0.3.28 as a workaround. All GPU code was compiled with `hipcc` (clang-based) targeting `gfx942`.

CPU benchmarks used the quimb tensor network library [6] (version 1.8) with cotengra [20] for automatic contraction path optimization, backed by OpenBLAS with thread counts of 1, 2, 4, 8, and 12.

4.2. Benchmark methodology

Each benchmark configuration specifies the implementation, physics model, system size L, bond dimension χ, and number of sweeps (or outer iterations for parallel DMRG). All runs use open boundary conditions. Wall times are measured from program start to convergence or sweep limit, including all initialization. Reported times are from the internal "Total wall time" timer, which excludes only process startup.

Correctness is validated against exact diagonalization energies for small systems ($L = 4$–8) and against converged quimb reference energies for larger systems, with a pass threshold of $|\Delta E| < 10^{-10}$. A total of 604 benchmark

configurations were run for the main comparison (all implementations $\times$ all models $\times$ all sizes), plus 108 configurations for the GPU optimization scaling study.

For CPU benchmarks, we report the best wall time across all thread counts (1, 2, 4, 8, 12) for each configuration, as the optimal thread count varies with problem size.

4.3. Physics models

We benchmark three Hamiltonians that exercise different aspects of the DMRG implementation:

Heisenberg spin-$\frac{1}{2}$ chain..

$$H = \sum_{i=1}^{L-1} \left(S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + S_i^z S_{i+1}^z \right), \tag{7}$$

with $d = 2$, real arithmetic, and MPO bond dimension $D = 5$. This is the standard DMRG benchmark [1], with the exact ground state energy density approaching $e_0 = 1/4 - \ln 2 \approx -0.4432$ per bond in the thermodynamic limit [21].

Josephson junction array..

$$H = -\frac{E_J}{2} \sum_i \left(e^{i\varphi_{\text{ext}}} e^{i\hat{\varphi}_i} e^{-i\hat{\varphi}_{i+1}} + \text{h.c.} \right) + E_C \sum_i \hat{n}_i^2, \tag{8}$$

with $E_J = 1.0$, $E_C = 0.5$, external flux $\varphi_{\text{ext}} = \pi/4$, and charge basis truncation $n_{\max} = 2$ giving $d = 5$ and complex arithmetic [22]. The MPO bond dimension is $D = 4$. The nonzero external flux breaks time-reversal symmetry, requiring full complex-valued MPS and MPO tensors.

Transverse-field Ising model (TFIM)..

$$H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i, \tag{9}$$

with $h/J = 1.0$ at the quantum critical point [23], $d = 2$, real arithmetic, and $D = 3$. At criticality, the entanglement entropy grows logarithmically with subsystem size, $S \sim (c/6) \ln L$ with central charge $c = 1/2$, providing a stress test for bond dimension convergence.

5. Results

5.1. CPU vs. GPU crossover

Table 1 summarizes the win rates between GPU and CPU implementations across all converged benchmark configurations, stratified by bond dimension. A "win" is assigned to the fastest implementation for each (L, χ, model) configuration, considering all GPU implementations and the best CPU result across thread counts.

Table 1: Win rates by bond dimension. D1-GPU: single-site GPU DMRG; D2-GPU: two-site GPU DMRG; Q-D1, Q-D2: quimb single-/two-site with best thread count. All Newton–Schulz/Davidson variants achieve 0% win rate and are omitted.

χ	D1-GPU	D2-GPU	Q-D1	Q-D2
20	25.0%	25.0%	37.5%	12.5%
50	26.7%	0%	60.0%	13.3%
128	80.0%	0%	20.0%	0%

At $\chi = 20$–50, CPU implementations (quimb with 4-thread OpenBLAS) win the majority of configurations. The individual GEMMs at $\chi = 50$ (matrices of size $\sim 50 \times 50$) fit entirely in CPU L2 cache, while on the GPU, kernel launch overhead ($\sim$5–10 μs per launch) across hundreds of launches per Lanczos iteration outweighs the compute.

The crossover depends on *both* bond dimension and system size. Table 2 stratifies the GPU vs. CPU win rate by L range and χ. At small L (≤ 20), the GPU already wins 67% of configurations even at $\chi = 20$, because short chains require few sweeps and the per-bond GPU advantage dominates. At medium L (32–64), the CPU wins every configuration at $\chi \leq 50$—the increased number of bonds amplifies kernel launch overhead—and the GPU recovers only at $\chi = 128$ (71%). At large L (≥ 100), even $\chi = 128$ yields only a 50% GPU win rate, because the many-sweep convergence behavior favors the CPU's lower per-iteration overhead at moderate matrix sizes.

5.1.1. Recommended CPU implementation

Among CPU implementations, quimb single-site DMRG (quimb-dmrg1) with 4-thread OpenBLAS dominates at all system sizes and bond dimensions, winning 83–100% of configurations across all (L, χ) cells (Table 3). The only exception is at large L (≥ 64) with $\chi = 50$, where quimb two-site DMRG

Table 2: GPU vs. CPU win rates stratified by system size range and bond dimension. Each cell shows GPU wins / total configurations (win rate). CPU wins are the complement.

L range	$\chi = 20$	$\chi = 50$	$\chi = 128$
8–20	4/6 (67%)	4/6 (67%)	6/6 (100%)
32–64	0/2 (0%)	0/7 (0%)	5/7 (71%)
≥ 100	—	0/2 (0%)	1/2 (50%)

wins half of the configurations—the same regime where two-site convergence advantages appear. Our custom Python PDMRG never wins. For CPU-only deployments, quimb single-site DMRG with modest thread parallelism (4 threads) is the clear recommendation.

Table 3: CPU implementation win rates by system size and bond dimension. Only converged runs included. Cells show quimb-dmrg1 / quimb-dmrg2 wins (total configs).

L range	$\chi = 20$	$\chi = 50$	$\chi = 128$
8–20	5/1 (6)	6/0 (6)	5/1 (6)
32–64	2/0 (2)	6/1 (7)	5/0 (5)
≥ 100	—	1/1 (2)	2/0 (2)

5.1.2. Recommended GPU implementation

The choice between single-site (dmrg-gpu) and two-site (dmrg2-gpu) on the GPU depends on both system size and bond dimension (Table 4). At $\chi \geq 50$, single-site wins the majority of configurations at all system sizes (71–100%). Single-site's advantage comes from the smaller local eigensolve (χd vs. χd^2), which is particularly pronounced for the Josephson model ($d = 5$, so $d^2 = 25$).

At $\chi = 20$, the picture reverses for medium-to-large systems: two-site wins 100% of $L = 32$–64 configurations because adaptive bond dimension growth converges in 2–3 sweeps vs. 5–10 for single-site, and the per-bond cost penalty from d^2 is modest at small χ. At large L (≥ 100) with $\chi = 128$, the split is 50/50—two-site's convergence advantage is balanced against its higher per-bond cost when many sweeps are needed.

Parallel DMRG (pdmrg-gpu) never wins on a single GPU.

The practical recommendation is: use single-site dmrg-gpu at $\chi \geq 50$

Table 4: GPU implementation win rates (single-site / two-site) by system size and bond dimension. Only converged runs with both implementations available.

L range	$\chi = 20$	$\chi = 50$	$\chi = 128$
8–20	3/3 (6)	5/1 (6)	6/0 (6)
32–64	0/2 (2)	5/2 (7)	6/1 (7)
≥ 100	—	1/1 (2)	1/1 (2)

for most problems. Switch to two-site dmrg2-gpu when convergence is slow ($L \geq 64$ with small d) or when χ is small (≤ 20) and the system is not tiny.

5.2. Single-site vs. two-site convergence

Two-site DMRG (dmrg2-gpu) converges substantially faster than single-site (dmrg-gpu) at large system sizes, because it can adapt the bond dimension during sweeps rather than requiring a pre-set χ profile.

Table 5: Serial GPU wall times (seconds) for Heisenberg OBC. Times include all sweeps to convergence.

L	χ	dmrg-gpu	dmrg2-gpu	dmrg-gpu-opt	dmrg2-gpu-opt
8	32	0.31	0.34	0.68	0.78
32	128	4.21	5.63	5.09	8.87
64	128	24.04	22.83	26.63	30.64
100	128	176.80	53.13	157.13	69.32

At $L = 100$, $\chi = 128$, dmrg2-gpu completes in 53 s vs. 177 s for dmrg-gpu (3.3× faster). This advantage comes from two-site optimization's ability to converge in 2–3 sweeps by dynamically growing the bond dimension, while single-site requires 5–10 sweeps at fixed χ. The "-opt" variants (Newton–Schulz + Block-Davidson) are uniformly slower, as analyzed in Section 5.4.

5.3. Parallel segment scaling

The parallel DMRG (pdmrg-gpu) partitions the chain into P segments with independent sweeps. Table 6 shows wall times for the optimized parallel implementation.

Parallel DMRG is slower than serial two-site DMRG (dmrg2-gpu) at all tested sizes. At $L = 64$, $\chi = 128$, pdmrg-gpu takes 49 s with $P = 2$ segments

Table 6: Parallel GPU DMRG wall times (seconds), Heisenberg OBC.

L	χ	$P = 2$	$P = 4$
8	32	0.31	—
20	50	2.54	—
32	128	16.46	—
64	128	49.38	78.11
100	128	216.46	196.52

vs. 23 s for dmrg2-gpu. The overhead comes from three sources: (i) warmup sweeps are serial single-site sweeps, (ii) segment sweeps disrupt the global MPS entanglement structure, requiring expensive full-chain coupling sweeps to restore, (iii) the coupling sweep is itself a full serial sweep.

Amdahl's law limits the potential speedup. At $L = 64$, $\chi = 128$, the serial fraction (warmup + coupling + polish) accounts for 22.1 s out of 24.1 s total, capping the theoretical speedup at 1.22× over serial DMRG even with zero-cost parallel segments.

5.4. Algorithmic variant performance

5.4.1. Newton–Schulz + Block-Davidson: 0% win rate

The Newton–Schulz variants (dmrg-gpu-opt, dmrg2-gpu-opt) lose to their baselines in every converged comparison: 0 wins out of 50 configurations (24 single-site, 26 two-site). Table 7 shows representative results.

Table 7: Algorithmic variant (opt) vs. baseline wall times (seconds). All models, converged runs only. Speedup < 1 means the variant is slower.

Model	L	χ	Type	Baseline	Opt
Heisenberg	16	20	1-site	1.7	3.5
Heisenberg	32	50	2-site	3.5	8.0
Heisenberg	64	20	1-site	16.6	55.1
Josephson	16	50	2-site	2.9	8.4
Josephson	32	256	1-site	18.4	104.6
TFIM	32	128	1-site	9.8	24.2
TFIM	128	50	2-site	28.1	39.3

The slowdown ranges from 1.4× (best case, TFIM $L = 128$ $\chi = 50$) to 5.7× (Josephson $L = 32$ $\chi = 256$).

At $\chi \geq 128$, Newton–Schulz diverges catastrophically for the Heisenberg model. The Frobenius-norm-scaled iteration (5) fails to converge when the condition number of the two-site tensor θ is large, producing MPS tensors with corrupted entries that cause the energy to diverge to $O(10^6)$–$O(10^{20})$. A fallback to LAPACK SVD is triggered when $\|X^\dagger X - I\|_F > 10^{-8}$, but by the time the fallback activates, the accumulated error from previous corrupted truncations has already destabilized the MPS.

5.4.2. Chebyshev-filtered subspace iteration: 1.9–11× *slower*

CheFSI with degree $p = 15$ and $n_{\text{outer}} = 20$ converges to the correct energy ($|\Delta E| < 10^{-13}$) but is dramatically slower than Lanczos:

Table 8: Chebyshev-filtered subspace iteration vs. Lanczos eigensolver (pdmrg-gpu-opt, Heisenberg, $P = 2$ segments).

L	χ	Lanczos (s)	CheFSI (s)	Slowdown
8	32	0.73	1.40	1.9×
20	50	5.27	59.15	11.2×

The root cause is a fundamental mismatch between algorithm and problem. CheFSI applies a degree-p polynomial filter for n_{outer} iterations, requiring up to $p \times n_{\text{outer}} = 300$ matrix–vector products. Lanczos converges in ∼15 iterations for DMRG's single ground state. CheFSI was designed for DFT calculations [18] where many eigenvalues are computed simultaneously, amortizing the polynomial cost across $O(N_e)$ eigenstates. For a single eigenvalue, Krylov methods are fundamentally more efficient: Lanczos achieves $O(\sqrt{\kappa})$ convergence that no polynomial filter can match.

5.4.3. Cross-segment batched GEMM: slower at 18/19 configurations

The batched sweep mode (Section 3.4) was tested across 19 configurations with $L = 20$–100, $\chi = 50$–256, and $P = 2$–16 segments. Table 9 shows representative results.

Only one configuration ($\chi = 256$, $P = 2$) shows a speedup (1.18×). The batched mode fails because: (i) the lock-step sweep serializes BLAS-1 operations (dot, nrm2, axpy) in the Lanczos eigensolver that run concurrently in the thread-per-segment baseline; (ii) segments often have mismatched bond dimensions, preventing effective GEMM batching; (iii) at $\chi = 256$ with only $P = 2$ segments, the individual GEMMs are large enough that batching the pointer array dispatch provides modest savings.

Table 9: Cross-segment batched GEMM sweep vs. thread-per-segment baseline (pdmrg-gpu-opt, Heisenberg).

L	χ	P	Baseline (s)	Batched (s)	Speedup
20	50	2	2.35	3.04	0.77×
32	128	2	16.55	231.81	0.07×
32	128	4	15.37	25.56	0.60×
32	256	2	59.91	50.66	**1.18**×
32	256	4	25.86	40.25	0.64×
64	128	2	72.23	timeout	—
100	128	4	—	196.52	—

5.4.4. Additive two-level parallel DMRG (A2DMRG)

The A2DMRG algorithm (Section 3.5) was tested on the Heisenberg model with $P = 2$ MPI ranks and 2 serial DMRG2 warm-up sweeps, followed by up to 20 A2DMRG iterations. Table 10 shows accuracy and wall times compared to serial quimb DMRG2.

Table 10: A2DMRG vs. serial DMRG2 (quimb) on the Heisenberg chain, $P = 2$, 2 warm-up sweeps. $|\Delta E|$ is the absolute energy error vs. converged reference.

L	χ	$\lvert\Delta E\rvert$	A2DMRG (s)	DMRG2 (s)	Status
8	20	3×10^{-15}	0.1	0.1	PASS
12	20	7×10^{-15}	0.2	0.1	PASS
12	50	7×10^{-15}	0.3	0.2	PASS
16	50	2×10^{-14}	0.7	0.4	PASS
20	50	5×10^{-11}	2.1	0.8	PASS
32	50	2.7×10^{-7}	7.4	5.2	FAIL
32	100	8.9×10^{-9}	35.6	11.4	WARN
48	50	3.1×10^{-6}	15.9	11.4	FAIL
48	100	1.9×10^{-5}	104.3	26.8	FAIL
64	50	5.7×10^{-5}	32.7	17.4	FAIL
64	100	1.9×10^{-5}	198.3	44.4	FAIL

A2DMRG achieves machine-precision accuracy ($|\Delta E| < 10^{-10}$) for $L \leq 20$, matching the system sizes tested in the original paper [19] ($d \leq 24$). However, accuracy degrades rapidly beyond $L \approx 30$: at $L = 64$, errors reach

10^{-5}, and the algorithm is 2–4× slower than serial DMRG2 even with only $P = 2$ ranks.

The degradation has a clear structural cause: the linear combination phase produces bond dimensions up to $(L-1)\times\chi$, which must be compressed back to χ via TT-SVD. At $L = 32$, $\chi = 20$, this is a 31:1 compression ratio, causing significant information loss. The original paper avoids this regime by testing quantum chemistry Hamiltonians with $d \leq 24$ and using a rank-growth strategy that starts with small bond dimensions and gradually increases them, yielding gentler compression ratios in early sweeps.

A diagnostic experiment reveals a deeper issue with the algorithm's practical utility. Table 11 shows how A2DMRG accuracy at $L = 32$ depends on the number of serial DMRG2 warm-up sweeps.

Table 11: A2DMRG accuracy vs. number of serial DMRG2 warm-up sweeps (Heisenberg $L = 32$, $\chi = 20$, $P = 4$).

Warm-up	$\lvert\Delta E\rvert$	Total time (s)	Note
0	1.27	74.8	Not converged
1	5.7×10^{-3}	0.2	—
2	4.4×10^{-6}	0.3	—
3	1.5×10^{-8}	0.3	—
5	5.2×10^{-12}	0.4	Machine precision
10	2.9×10^{-13}	0.4	—

Each additional warm-up sweep buys 2–3 orders of magnitude in accuracy. By 5 warm-up sweeps, A2DMRG reaches machine precision—but the A2DMRG phase itself then converges in 1–2 iterations taking < 0.1 s, because the serial warm-up has already produced a nearly-converged state. This creates a paradox: sufficient warm-up makes the algorithm accurate but also makes the parallel phase redundant, since serial DMRG2 alone would have converged in the same time.

We also tested A2DMRG on a complex-valued Bose–Hubbard/Josephson Hamiltonian ($d = 5$, $L = 16$), where it failed catastrophically ($|\Delta E| \sim 10^3$). The original paper tests only real-valued quantum chemistry Hamiltonians, and the complex eigensolver and TT-SVD compression path may require additional stabilization for non-Hermitian effective Hamiltonians.

Three important caveats apply to our A2DMRG results. First, the original paper reports theoretical per-processor FLOP counts, not wall-clock

times, assuming $d-1$ processors (one per site); our $P=2$ tests are in the algorithm's worst operating regime. Second, quantum chemistry Hamiltonians have all-to-all MPO structure that may favor the coarse-space correction more than the constant-D MPOs of lattice models. Third, the rank-growth initialization strategy used in the original paper avoids the large compression ratios that degrade our cold-start results.

6. Discussion

6.1. Why BLAS-3 replacements fail at moderate χ

The central negative finding of this work is that replacing BLAS-2 algorithms (Lanczos, Householder QR for SVD) with BLAS-3 alternatives (Newton–Schulz, Davidson, Chebyshev) does not improve DMRG performance at $\chi \leq 256$ on the MI300X. This contradicts the natural expectation that GPU hardware optimized for matrix multiplication should benefit from a higher GEMM fraction.

The failure has three causes:

LAPACK SVD is highly optimized.. LAPACK's QR-iteration SVD (`dgesvd`) performs a single $O(n^3)$ factorization with low constant factors, exploiting decades of numerical linear algebra optimization [24]. Newton–Schulz replaces this with up to 30 GEMM iterations at $O(n^3)$ each, plus a CPU-side eigendecomposition. The total FLOP count is up to 30× higher, and the GEMM throughput advantage on the MI300X does not compensate at $n = \chi \leq 256$. At $\chi = 128$, the matrices are 256×256 (two-site) — far below the $\sim$2048 $\times$ 2048 threshold where the MI300X's MFMA units reach peak occupancy.

Lanczos is near-optimal for a single eigenvalue.. For finding the smallest eigenvalue of the $\chi^2 d^2 \times \chi^2 d^2$ effective Hamiltonian, Lanczos converges in $O(\sqrt{\kappa})$ matrix–vector products [25], where κ is the condition number. In practice, this means 10–50 iterations for DMRG. Block-Davidson adds subspace management overhead (QR restarts, projected eigensolves) without reducing the iteration count. CheFSI requires $O(p \cdot n_{\text{outer}})$ matrix–vector products regardless of the spectral gap, making it fundamentally wasteful for a single eigenvalue.

DMRG matrices are small by GPU standards.. The MI300X's 304 compute units are designed for large-scale parallelism. At $\chi = 128$, the effective Hamiltonian operates on vectors of dimension $\chi^2 d^2 = 65{,}536$ (Heisenberg, two-site) with matrices of size $\sim 256 \times 256$. The GEMM operations in each Lanczos iteration launch thousands of wavefronts but each GEMM completes in microseconds, making launch overhead the dominant cost rather than arithmetic throughput.

6.2. CPU SVD bottleneck

Profiling of dmrg2-gpu at $L = 64$, $\chi = 256$ reveals the wall-time breakdown per sweep:

Table 12: Per-sweep wall-time breakdown for dmrg2-gpu, Heisenberg $L = 64$, $\chi = 256$.

Component	Time	Fraction
SVD (CPU LAPACK)	38–41 s	97–98%
Lanczos + H_{eff}	0.4–0.6 s	1–2%
Environment updates	0.03 s	$< 0.1\%$

The SVD operates on matrices of size $(\chi d, d\chi) = (512, 512)$ at $\chi = 256$ for the Heisenberg model. Each LAPACK `dgesvd` call takes $\sim$300 ms, and $\sim$126 SVDs per sweep (63 bonds $\times$ 2 directions) total $\sim$38 s.

This profile explains why all non-SVD optimizations have negligible impact: even if the Lanczos eigensolver and environment updates were made infinitely fast, the wall time would decrease by at most 2–3%.

GPU SVD via rocsolver provides a modest 13% improvement at $\chi = 256$ ($L = 64$: 124 s vs. 142 s for 3 sweeps), but paradoxically makes parallel DMRG *slower* because the smaller effective bond dimensions within segments ($\chi_{\text{eff}} \approx 50$–60) put the GPU SVD in a regime where its overhead exceeds the CPU's.

6.3. Implications for future GPU DMRG

Our results identify the conditions under which GPU DMRG and its algorithmic variants would be expected to provide meaningful speedups:

Large bond dimensions ($\chi \gtrsim 1024$).. At large χ, the GEMM operations in H_{eff} application become large enough to saturate the GPU's compute units, and iterative methods like Newton–Schulz may amortize their iteration overhead against the LAPACK SVD's $O(\chi^3)$ cost. However, the numerical stability of Newton–Schulz at large condition numbers remains a concern.

Multi-GPU parallelism.. The real-space parallel DMRG algorithm [13] was designed for distributed-memory systems with one segment per processor. On a single GPU, the segment parallelism competes for the same compute resources. With multiple GPUs, each segment would have dedicated compute capacity, and the Amdahl's law bottleneck from serial coupling sweeps could be addressed through pipelining.

Alternative SVD approaches.. Randomized SVD [26] could provide speedups when the truncation rank is much smaller than χ, but in DMRG the truncation typically retains $\chi_{\max}$ singular values, limiting the savings. Mixed-precision approaches (single-precision for warmup, double for final convergence) could halve the SVD time for the majority of sweeps.

7. Conclusions

We have presented a systematic benchmark of eleven DMRG implementations on the AMD MI300X GPU, spanning three physics models and a range of system sizes and bond dimensions. Our main findings are:

1. The GPU vs. CPU crossover depends on both χ and L. At $\chi = 128$, the GPU wins 100% of short-chain ($L \leq 20$) configurations but only 71% at $L = 32$–64 and 50% at $L \geq 100$. At $\chi \leq 50$, the CPU wins all medium-to-large systems ($L \geq 32$).
2. Two-site DMRG is $\sim$3$\times$ faster than single-site at large systems ($L = 100$) due to faster convergence from adaptive bond dimension growth.
3. Replacing BLAS-2 algorithms (Lanczos, LAPACK SVD) with BLAS-3 alternatives (Newton–Schulz polar decomposition, Block-Davidson, Chebyshev filtering) uniformly fails to improve performance at $\chi \leq 256$. The Newton–Schulz variant achieves a 0% win rate across 50 configurations and diverges numerically at $\chi \geq 128$. Chebyshev filtering is 1.9–11$\times$ slower. Cross-segment batched GEMM is slower in 18 of 19 configurations.

4. CPU SVD consumes 97–98% of per-sweep wall time at $\chi = 256$. This bottleneck cannot be addressed by any optimization that targets the remaining 2–3% of runtime.
5. Parallel DMRG on a single GPU is slower than serial two-site DMRG at all tested sizes, limited by the serial coupling sweep required to maintain global MPS consistency.
6. The additive two-level parallel DMRG (A2DMRG) algorithm [19] achieves machine-precision accuracy for $L \leq 20$ but degrades to 10^{-5} errors at $L = 64$ due to TT-SVD compression loss. With $P = 2$ ranks, it is 2–4× slower than serial DMRG2, and sufficient serial warm-up renders the parallel phase redundant. This is the first independent reproduction of the algorithm and the first test on lattice spin models.

These negative results delineate the boundary between problem regimes where GPU algorithmic variants can and cannot compete with mature CPU linear algebra. At moderate bond dimensions ($\chi \leq 256$), LAPACK's optimized SVD and the Lanczos algorithm's efficiency for single-eigenvalue problems make them difficult targets. The regime where BLAS-3 replacements would be expected to help—$\chi \gtrsim 1024$ on multi-GPU systems—remains to be explored.

All source code (11 implementations in Python, Python/MPI, and C++/HIP), benchmark scripts, and raw benchmark data are available at `https://github.com/yye00/dmrg-implementations`.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code, benchmark scripts, and raw benchmark data are available at `https://github.com/yye00/dmrg-implementations`. The repository includes build instructions for ROCm 7.2 on AMD MI300X (gfx942). Benchmark results are stored in JSON format in the `benchmarks/paper_results/` directory, comprising 604 entries in the main dataset (`results.json`) and 108 entries in the GPU optimization study (`gpu_opt_bench.json`).

Acknowledgments

We acknowledge access to AMD Instinct MI300X hardware for benchmarking.

References

[1] S. R. White, Density matrix formulation for quantum renormalization groups, Physical Review Letters 69 (1992) 2863–2866. doi:10.1103/PhysRevLett.69.2863.

[2] S. R. White, Density-matrix algorithms for quantum renormalization groups, Physical Review B 48 (1993) 10345–10356. doi:10.1103/PhysRevB.48.10345.

[3] U. Schollwöck, The density-matrix renormalization group in the age of matrix product states, Annals of Physics 326 (2011) 96–192. doi:10.1016/j.aop.2010.09.012.

[4] R. Orús, A practical introduction to tensor networks: Matrix product states and projected entangled pair states, Annals of Physics 349 (2014) 117–158. doi:10.1016/j.aop.2014.06.013.

[5] M. Fishman, S. R. White, E. M. Stoudenmire, The ITensor software library for tensor network calculations, SciPost Physics Codebases (2022) 4doi:10.21468/SciPostPhysCodeb.4.

[6] J. Gray, quimb: A Python package for quantum information and many-body calculations, Journal of Open Source Software 3 (2018) 819. doi:10.21105/joss.00819.

[7] C. Nemes, G. Barcza, Z. Nagy, Ö. Legeza, P. Szolgay, The density matrix renormalization group algorithm on kilo-processor architectures: Implementation and trade-offs, Computer Physics Communications 185 (2014) 1570–1581. doi:10.1016/j.cpc.2014.02.surface.

[8] J. Eisert, M. Cramer, M. B. Plenio, Colloquium: Area laws for the entanglement entropy, Reviews of Modern Physics 82 (2010) 277–306. doi:10.1103/RevModPhys.82.277.

[9] I. V. Oseledets, Tensor-train decomposition, SIAM Journal on Scientific Computing 33 (2011) 2295–2317. doi:10.1137/090752286.

[10] S. R. White, Density matrix renormalization group algorithms with a single center site, Physical Review B 72 (2005) 180403. doi:10.1103/PhysRevB.72.180403.

[11] Advanced Micro Devices, rocBLAS: ROCm basic linear algebra subprograms, https://rocm.docs.amd.com/projects/rocBLAS/ (2024).

[12] Advanced Micro Devices, ROCm documentation, https://rocm.docs.amd.com/, version 7.2 (2024).

[13] E. M. Stoudenmire, S. R. White, Real-space parallel density matrix renormalization group, Physical Review B 87 (2013) 155137. doi:10.1103/PhysRevB.87.155137.

[14] G. Schulz, Iterative Berechnung der reziproken Matrix, Zeitschrift für Angewandte Mathematik und Mechanik 13 (1933) 57–59. doi:10.1002/zamm.19330130111.

[15] Y. Nakamura, N. J. Higham, Optimized stable scaled Newton–Schulz iteration for the matrix sign function and polar decomposition, SIAM Journal on Matrix Analysis and Applications (2020).

[16] E. R. Davidson, The iterative calculation of a few of the lowest eigenvalues and corresponding eigenvectors of large real-symmetric matrices, Journal of Computational Physics 17 (1975) 87–94. doi:10.1016/0021-9991(75)90065-0.

[17] B. Liu, The simultaneous expansion method for the iterative solution of several of the lowest eigenvalues and corresponding eigenvectors of large real-symmetric matrices, Numerical Algorithms in Chemistry: Algebraic Methods 49 (1978).

[18] Y. Zhou, Y. Saad, M. L. Tiago, J. R. Chelikowsky, Self-consistent-field calculations using Chebyshev-filtered subspace iteration, Journal of Computational Physics 219 (2006) 172–184. doi:10.1016/j.jcp.2006.03.017.

[19] L. Grigori, S. B. Hassan, An additive two-level parallel algorithm for the density matrix renormalization group method, arXiv preprint-ArXiv:2505.23429v2 (2025). arXiv:2505.23429.

[20] J. Gray, S. Kourtis, Hyper-optimized tensor network contraction, Quantum 5 (2021) 410. doi:10.22331/q-2021-03-15-410.

[21] H. Bethe, Zur Theorie der Metalle. I. Eigenwerte und Eigenfunktionen der linearen Atomkette, Zeitschrift für Physik 71 (1931) 205–226. doi:10.1007/BF01341708.

[22] R. Fazio, H. van der Zant, Quantum phase transitions and vortex dynamics in superconducting networks, Physics Reports 355 (2001) 235–334. doi:10.1016/S0370-1573(01)00022-9.

[23] P. Pfeuty, The one-dimensional Ising model with a transverse field, Annals of Physics 57 (1970) 79–90. doi:10.1016/0003-4916(70)90270-8.

[24] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, D. Sorensen, LAPACK Users' Guide, 3rd Edition, SIAM, 1999.

[25] C. Lanczos, An iteration method for the solution of the eigenvalue problem of linear differential and integral operators, Journal of Research of the National Bureau of Standards 45 (1950) 255–282.

[26] N. Halko, P.-G. Martinsson, J. A. Tropp, Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, SIAM Review 53 (2011) 217–288. doi:10.1137/090771806.

Appendix A. Complete benchmark results

Tables A.13–A.15 report wall times (seconds) for all converged benchmark configurations. For CPU implementations (Q-D1: quimb-dmrg1, Q-D2: quimb-dmrg2), the reported time is the best across thread counts (1, 2, 4, 8, 12). For GPU implementations (D1-GPU: dmrg-gpu, D2-GPU: dmrg2-gpu, PD-GPU: pdmrg-gpu with $P = 2$ segments), all runs use a single MI300X with CPU SVD. The fastest implementation for each configuration is shown

Table A.13: Complete wall times (seconds) for the Heisenberg spin-$\frac{1}{2}$ chain (OBC, $d = 2$, $D = 5$, real). Bold indicates fastest.

L	χ	Q-D1	Q-D2	D1-GPU	D2-GPU	PD-GPU
12	20	0.81	1.10	0.67	**0.52**	0.71
12	50	0.78	1.06	**0.42**	0.56	0.65
12	128	0.83	0.99	**0.42**	0.54	0.66
20	20	1.11	1.12	2.31	**1.07**	1.89
20	50	**1.08**	1.43	1.42	1.45	2.26
20	128	2.05	4.82	**1.36**	2.36	3.36
32	20	**1.50**	1.69	5.06	2.27	—
32	50	**1.82**	2.91	6.20	2.91	—
32	128	5.24	15.24	**4.24**	5.67	—
64	50	5.67	**5.05**	26.99	9.27	24.50
64	128	**20.59**	37.69	24.01	22.80	—
100	50	11.36	**8.50**	74.22	43.09	—
100	128	**40.70**	75.68	149.40	53.57	—

in **bold**. A dash (—) indicates the implementation was not tested or did not converge for that configuration.

Several patterns emerge from the full results:

- **GPU single-site (D1-GPU) dominates the TFIM**: it wins 10 of 13 TFIM configurations, including all of $L \leq 20$. The TFIM's small MPO bond dimension ($D = 3$) and fast convergence (2–3 sweeps) keep per-bond costs low, favoring the GPU even at moderate χ.

- **Josephson is hardest for the GPU**: complex arithmetic doubles the GEMM cost, and $d = 5$ inflates the two-site eigensolve dimension to $\chi d^2 = 25\chi$. D2-GPU is never the fastest for Josephson at $\chi \geq 50$. The CPU (Q-D1) wins all Josephson configurations at $\chi = 50$, and the GPU only overtakes at $\chi = 128$ for $L \leq 32$.

- **Parallel DMRG (PD-GPU) never wins**: across all 28 configurations where it was tested, pdmrg-gpu is always slower than both the best serial GPU and best CPU implementation. Its closest approach to competitiveness is Heisenberg $L = 12$, $\chi = 20$ (0.71 s vs. 0.52 s for D2-GPU), but even there it loses. The overhead of warmup, coupling,

Table A.14: Complete wall times (seconds) for the Josephson junction array (OBC, $d = 5$, $D = 4$, complex). Bold indicates fastest.

L	χ	Q-D1	Q-D2	D1-GPU	D2-GPU	PD-GPU
8	20	1.46	**1.31**	2.67	1.49	4.70
8	50	1.25	3.18	**1.10**	1.16	19.05
8	128	3.55	6.52	**0.97**	2.54	20.01
16	20	**2.30**	3.53	6.19	6.38	12.40
16	50	**3.73**	16.32	10.70	8.03	55.61
16	128	26.19	228.10	**4.67**	29.17	314.34
32	50	**17.07**	59.69	32.30	126.23	239.93
32	128	107.98	891.90	**33.02**	107.87	800.42
48	50	**31.00**	360.51	64.69	244.94	326.85
48	128	**172.77**	—	191.16	1347.92	—
64	50	**41.62**	1287.38	88.55	568.37	—
64	128	300.79	—	**295.82**	981.48	—

and polish sweeps exceeds any benefit from parallel segment execution on a single GPU.

- **Large Heisenberg systems favor CPU or two-site GPU**: at $L \geq 64$, the CPU (Q-D1 or Q-D2) wins at $\chi \leq 128$. When the GPU wins (e.g., $L = 32$, $\chi = 128$), it is always D1-GPU. D2-GPU is competitive at $L = 64$, $\chi = 128$ (22.80 s vs. 24.01 s for D1-GPU) but does not win.

Table A.15: Complete wall times (seconds) for the transverse-field Ising model at criticality (OBC, $d = 2$, $D = 3$, real). Bold indicates fastest.

L	χ	Q-D1	Q-D2	D1-GPU	D2-GPU	PD-GPU
12	20	0.77	0.79	**0.39**	0.51	5.18
12	50	0.78	0.80	**0.41**	0.49	4.54
12	128	0.83	0.84	**0.41**	0.52	1.38
20	20	0.88	1.00	**0.69**	1.01	12.41
20	50	1.03	1.20	**0.98**	1.76	16.46
20	128	1.95	1.87	**1.52**	3.47	13.48
32	20	**1.14**	1.26	1.50	1.50	18.71
32	50	**1.64**	1.83	1.78	3.78	29.75
32	128	5.81	6.17	**4.89**	13.46	45.78
64	50	**3.97**	4.80	4.96	8.67	27.36
64	128	22.11	26.59	**20.45**	51.75	45.55
100	50	**7.66**	9.31	8.36	15.97	59.89
100	128	40.81	55.31	**36.31**	96.98	—